SALESIQ: AN INTELLIGENT E-COMMERCE SALES ANALYTICS AND INVENTORY MANAGEMENT SYSTEM

Samsung Innovation Campus- Coding & Programming Course

Submitted by

SHIVANGI TRIPATHI (RA2311026011081)
ANGIRA YADAV (RA2311003011468)
MADHUMITHA T (RA2311027010174)

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE ENGINEERING

COLLEGE OF ENGINEERING AND TECHNOLOGY
SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
KATTANKULATHUR- 603 203

ABSTRACT

The rapid growth of e-commerce platforms has resulted in the generation of large volumes of sales data every day. However, raw sales data often contains missing values, duplicate records, inconsistent formats, and invalid entries, making it difficult to derive meaningful business insights. Manual analysis of such data is time-consuming and prone to errors, highlighting the need for an automated and intelligent sales analytics system.

This project, **SalesIQ: An Intelligent E-Commerce Sales Analytics and Inventory Management System**, presents an end-to-end solution that integrates data engineering, data structures, analytics, and visualization into a single platform. The system begins with a synthetic e-commerce dataset that simulates real-world sales transactions. An **ETL (Extract, Transform, Load) pipeline** is implemented to clean, validate, and transform the raw data by handling missing values, removing duplicate records, standardizing data formats, and generating derived business metrics such as total sales and net revenue.

To improve inventory management, the project incorporates a **custom Min-Heap data structure**, which efficiently prioritizes products based on stock availability, enabling faster identification of items that require replenishment. Business analytics are then performed to generate insights including product-wise sales summaries, revenue trends, regional sales distribution, and fraud detection based on predefined business rules.

The processed data is exposed through a **FastAPI-based REST API** and visualized using an interactive dashboard developed with HTML, CSS, and JavaScript. Additional visualizations such as charts and an interactive regional sales map provide a comprehensive understanding of business performance and support data-driven decision-making.

Overall, the proposed system demonstrates how data engineering, efficient algorithms, and business intelligence techniques can be integrated to build a scalable and user-friendly e-commerce sales analytics solution. The project offers a practical approach for improving operational efficiency, inventory planning, and strategic decision-making in modern e-commerce environments.

CHAPTER 1 INTRODUCTION

1.1 Introduction to Project

The e-commerce industry has experienced significant growth in recent years, resulting in a substantial increase in online transactions and sales data. Every purchase made by a customer generates valuable information that can be used to understand customer behavior, monitor sales performance, manage inventory, and support business decisions. However, raw sales data often contains inconsistencies such as missing values, duplicate records, invalid entries, and formatting errors, making accurate analysis challenging.

To address these challenges, businesses require an automated system capable of cleaning, processing, and analyzing sales data efficiently. Such a system should not only improve data quality but also generate meaningful insights that help organizations optimize inventory, identify sales trends, detect suspicious transactions, and enhance overall operational efficiency.

SalesIQ is an intelligent e-commerce sales analytics and inventory management system developed to provide a complete solution for these requirements. The project integrates an ETL (Extract, Transform, Load) pipeline to preprocess raw sales data, a custom Min-Heap data structure to prioritize inventory replenishment, business analytics to generate valuable insights, and an interactive dashboard for data visualization. Additionally, a FastAPI backend enables seamless communication between the processed data and the frontend dashboard.

1.2 Problem Statement

E-commerce businesses generate a large volume of sales data every day. However, this data often contains missing values, duplicate records, inconsistent formats, and invalid transactions, making analysis difficult and unreliable. Manual processing is time-consuming and prone to errors, while inefficient inventory management can lead to stock shortages or overstocking. Therefore, there is a need for an automated system that can clean, process, analyze, and visualize sales data. Therefore, there is a need for an automated system that can clean, process, analyze, and visualize sales data while efficiently managing inventory and supporting business decision-making

CHAPTER 2
SYSTEM ARCHITECTURE

2.1 Introduction

The proposed system follows a modular architecture that integrates data engineering, data structures, business analytics, backend services, and visualization into a unified platform. The system begins with the generation of a synthetic e-commerce sales dataset that simulates real-world online shopping transactions. This raw dataset undergoes an ETL (Extract, Transform, Load) process, where missing values, duplicate records, inconsistent formats, and invalid entries are identified and corrected to improve data quality.

Once the dataset is cleaned and transformed, it is processed by the analytics module to generate valuable business insights such as product-wise sales summaries, revenue analysis, regional sales distribution, and fraud detection. Simultaneously, the inventory management module utilizes a custom Min-Heap data structure to prioritize products that require immediate replenishment based on their stock levels.

The processed data is then stored in structured formats such as CSV and JSON files, which serve as the primary data source for the backend services. A FastAPI server provides RESTful API endpoints that allow the frontend dashboard to retrieve processed data dynamically. Finally, the interactive dashboard presents the generated insights through charts, tables, and geographical visualizations, enabling users to monitor sales performance and inventory status efficiently.

This modular architecture ensures that each component performs a specific function while maintaining seamless communication with other modules, resulting in a scalable, maintainable, and efficient e-commerce sales intelligence system.

2.2 Working of the system

The working of the proposed system consists of six major stages. Initially, a synthetic dataset containing approximately one thousand e-commerce transactions is generated to simulate real business operations. This dataset includes customer details, product information, pricing, discounts, payment methods, and regional sales information.

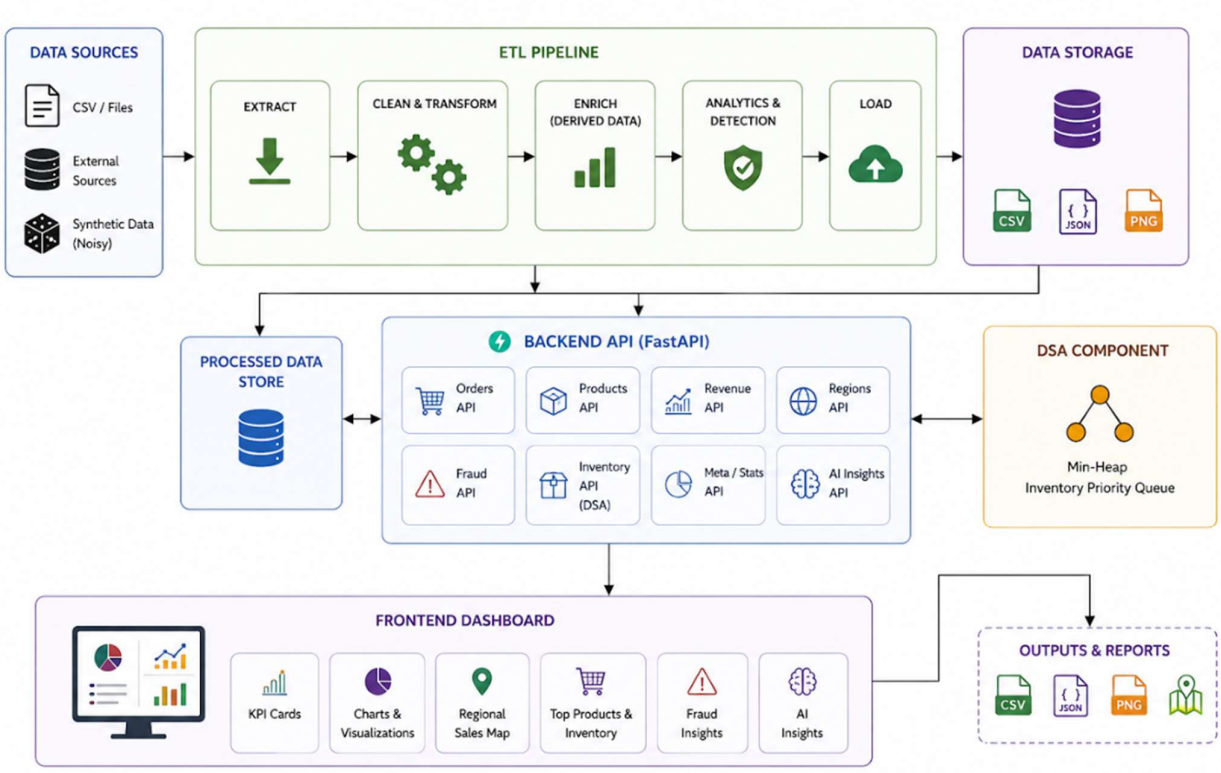
The generated dataset is then passed through the ETL pipeline, where artificial noise such as missing values, duplicate records, inconsistent currency formats, and invalid discount values are introduced to mimic real-world business scenarios. During the transformation stage, the system removes duplicate entries, handles missing values, validates business rules, standardizes data formats, and creates additional features such as total sales and net revenue.

After preprocessing, the cleaned dataset is utilized by the inventory management module, where a custom Min-Heap algorithm efficiently identifies products with the lowest stock levels, enabling faster inventory replenishment decisions.

The processed data is simultaneously analyzed by the business analytics module to generate meaningful insights including revenue analysis, product performance, regional sales distribution, and fraud detection. The generated analytical results are exported into CSV and JSON formats for easy integration with other modules.

A FastAPI backend exposes these processed datasets through RESTful API endpoints, allowing the frontend application to request data dynamically. Finally, the dashboard displays interactive charts, product summaries, fraud reports, and regional sales maps, providing users with a comprehensive view of business performance and supporting informed decision-making.

2.3 System Architecture Diagram



CHAPTER 3

TASK 1: DATA GENERATION AND ETL PIPELINE

3.1 Introduction

The first task of the proposed system focuses on generating and preprocessing e-commerce sales data through an ETL (Extract, Transform, Load) pipeline. In any data-driven application, the quality of the data directly influences the accuracy of analysis and decision-making. However, raw business data collected from online transactions is often incomplete, inconsistent, and contains duplicate or erroneous records. Such data cannot be directly used for analytics or visualization.

To overcome these challenges, an ETL pipeline was implemented as the foundation of the proposed system. The pipeline begins with the generation of a synthetic e-commerce sales dataset that simulates real-world customer transactions. The generated dataset is then processed through multiple stages including noise injection, data cleaning, transformation, feature engineering, and data export. These processes ensure that the final dataset is accurate, consistent, and suitable for inventory management, business analytics, fraud detection, and dashboard visualization.

The implementation of the ETL pipeline not only improves data quality but also demonstrates real-world data engineering practices followed in modern e-commerce organizations.

3.2 Subtask 1 – Synthetic Dataset Generation

The first stage of the ETL pipeline involves generating a synthetic e-commerce sales dataset. Since real business datasets are often confidential and unavailable for public use, a synthetic dataset was created using Python. The generated dataset closely resembles the transaction records maintained by an online shopping platform.

Approximately **1,000 transaction records** were generated, where each record represents a unique customer purchase. The dataset contains attributes such as Order ID, Customer ID, Product Name, Category, Quantity, Unit Price, Discount, Region, Order Date, Payment Method, Currency, and Fraud Status. These attributes provide sufficient information for performing sales analysis, inventory management, and fraud detection.

Random values were generated using Python's built-in **Random** module to ensure diversity among products, prices, quantities, regions, and payment methods. The generated records were then converted into a Pandas DataFrame, allowing efficient manipulation and preprocessing during later stages.

The generated dataset serves as the input to the ETL pipeline and forms the basis for all subsequent analytical operations performed in the project.

Python Commands Used

Command	Purpose
`import random`	Generates random values for dataset attributes.
`random.choice()`	Selects random products, regions, categories, and payment methods.
`random.randint()`	Generates random integer values such as quantity.
`random.uniform()`	Generates decimal values for prices and discounts.
`pd.DataFrame()`	Converts generated records into a tabular DataFrame.

3.3 Subtask 2 – Noise Injection

In real-world business environments, collected data is rarely perfect. Data may contain missing values, duplicate records, inconsistent formats, and invalid entries due to human errors, system failures, or communication issues. To simulate these practical scenarios, artificial noise was intentionally introduced into the generated dataset.

3.3.1 Missing Values

Missing values represent incomplete information where one or more fields in a transaction are left blank or unavailable. Such situations commonly occur due to network failures, incomplete customer forms, or interrupted transactions.

In this project, selected values were replaced with NaN (Not a Number) using NumPy to simulate missing information. This enables the ETL pipeline to demonstrate methods for handling incomplete records during preprocessing.

Python Commands Used

Command	Purpose
`np.nan`	Represents missing values in the dataset.
`loc[]`	Updates specific rows and columns with missing values.

3.3.2 Duplicate Records

Duplicate transactions occur when the same order is stored multiple times due to payment retries, synchronization failures, or software errors. Duplicate records lead to incorrect revenue

calculations and misleading business reports.

To simulate this situation, duplicate rows were intentionally inserted into the dataset. These duplicates are later identified and removed during the cleaning stage.

Python Commands Used

Command	Purpose
`pd.concat()`	Combines duplicate records with the original dataset.

3.3.3 Invalid Discount Values

Business rules specify that discount percentages should remain within a valid range (0–100%). However, incorrect data entry or import errors may produce invalid values such as 120% or 150%.

To demonstrate business rule validation, several records were assigned invalid discount values. These values are corrected during data preprocessing.

Python Commands Used

Command	Purpose
`loc[]`	Identifies and updates records with invalid discount values.

3.3.4 Currency Standardization

Sales data collected from different sources may contain varying currency formats such as **₹**, **Rs**, or **INR**. These inconsistencies can affect data processing and reporting.

To simulate this issue, multiple currency formats were introduced into the dataset. During preprocessing, all currency representations are standardized into a single format to maintain consistency.

Python Commands Used

Command	Purpose
`str.replace()`	Replaces inconsistent currency symbols with a standard format.

3.4 Subtask 3 – Data Cleaning

Data cleaning is an essential step in the ETL pipeline that improves the quality and reliability of the dataset. Since the generated dataset intentionally contains missing values, duplicate records, invalid discounts, and inconsistent formats, these issues must be resolved before analysis. Poor-quality data can lead to incorrect business insights and inaccurate decision-making.

The cleaning process begins by identifying missing values and either replacing them with

appropriate values or removing incomplete records, depending on their importance. Duplicate transactions are then detected and removed to ensure that each customer order is counted only once. Business rules are also applied to validate fields such as discount percentages, ensuring that values remain within an acceptable range. Finally, inconsistent text and currency formats are standardized so that similar values are represented uniformly throughout the dataset.

After completing these operations, the dataset becomes clean, consistent, and ready for further transformation and analysis.

Python Commands Used

Command	Purpose
`fillna()`	Fills missing values.
`dropna()`	Removes records with missing values.
`drop_duplicates()`	Removes duplicate records.
`loc[]`	Updates invalid values based on conditions.
`str.replace()`	Standardizes text and currency formats.

3.5 Subtask 4 – Data Transformation and Feature Engineering

Once the dataset is cleaned, the next step is to transform the data into a format suitable for business analysis. During this stage, new columns are created by performing calculations on the existing data. These additional features provide meaningful business insights and improve the effectiveness of the analytics module.

For example, the **Total Sales** value is calculated by multiplying the product quantity with the unit price. Similarly, **Net Revenue** is calculated after applying discounts to the total sales amount. Transactions that satisfy predefined business conditions are also marked with a fraud flag, enabling the system to identify suspicious activities. These derived attributes make the dataset more informative and support advanced sales analysis and reporting.

Feature engineering transforms raw transactional data into valuable business information that can be directly used for inventory management, visualization, and decision-making.

Python Commands Used

Command	Purpose
`df["Column"]`	Creates a new column.
Arithmetic Operators (`*` , `-`)	Calculate Total Sales and Net Revenue.
`apply()`	Applies functions to each row.
`lambda`	Creates custom conditions for fraud detection.

3.6 Subtask 5 – Data Export

After completing the cleaning and transformation processes, the final dataset is exported for use in different modules of the project. Storing the processed data in standard file formats ensures that it can be easily accessed by the analytics engine, backend APIs, and dashboard.

The cleaned dataset is exported as a **CSV file**, which serves as a structured data source for reporting and analysis. In addition, the processed information is converted into **JSON format**, allowing the FastAPI backend to efficiently transfer data to the frontend dashboard. These exported files act as the bridge between the ETL pipeline and the visualization modules, ensuring smooth data flow throughout the system.

By exporting the processed dataset, the project separates data preprocessing from visualization, making the architecture modular, scalable, and easier to maintain.

Python Commands Used

Command	Purpose
`to_csv()`	Exports the processed dataset as a CSV file.
`to_json()`	Converts the dataset into JSON format for the dashboard.

CHAPTER 4

TASK 2: INVENTORY MANAGEMENT USING MIN-HEAP

4.1 Introduction

Efficient inventory management is an important aspect of every e-commerce business. Products with low stock must be identified quickly so that they can be replenished before they become unavailable. If inventory is not managed properly, businesses may experience stock shortages, delayed deliveries, and dissatisfied customers.

To address this problem, the proposed system implements a **Min-Heap** data structure for inventory prioritization. A Min-Heap is a binary tree in which the smallest element is always stored at the root. In this project, the stock quantity of each product is used as the priority value. Products with the lowest stock automatically move to the top of the heap, allowing the system to identify the next product that requires restocking without searching the entire inventory.

The use of a Min-Heap improves the efficiency of inventory operations and demonstrates the practical application of data structures in solving real-world business problems.

4.2 Subtask 1 – Heap Creation

The first step in implementing inventory management is creating the Min-Heap. An empty heap is initialized to store product information along with their available stock quantities. Every product is represented as a node in the heap, where the stock quantity acts as the priority. The heap maintains the smallest stock quantity at the root, ensuring that products requiring immediate replenishment are always given the highest priority.

The heap is implemented as a list in Python, making insertion and deletion operations efficient while maintaining the heap property.

Python Commands Used

Command	Purpose
`class`	Creates the Min-Heap class.
`__init__()`	Initializes the heap.
`self.heap = []`	Creates an empty heap.

4.3 Subtask 2 – Inserting Products into the Heap

Each product is inserted into the heap along with its stock quantity. Initially, the new product is added at the end of the heap. After insertion, the heap property is checked. If the newly inserted product has a lower stock quantity than its parent, both elements are exchanged. This process

continues until the smallest stock value reaches its correct position.

This operation ensures that the product with the lowest inventory level always remains at the root of the heap, allowing quick identification of products that require replenishment.

4.4 Subtask 3 – Heapify Operation

The Heapify operation maintains the Min-Heap property after every insertion or deletion. Whenever the heap structure changes, the algorithm compares parent and child nodes and performs the required swaps until the smallest element reaches the root.

During insertion, the system performs **Heapify-Up**, whereas after removing the minimum element, **Heapify-Down** is executed. These operations ensure that the inventory remains correctly prioritized at all times without requiring the entire dataset to be sorted.

Python Commands Used

Command	Purpose
`while`	Repeats comparisons.
`if`	Checks heap condition.
`(i-1)//2`	Finds parent index.
`2*i+1`	Finds left child.
`2*i+2`	Finds right child.

4.5 Subtask 4 – Product Prioritization

Once the heap is constructed, the product with the lowest stock quantity is always available at the root. Whenever the warehouse manager needs to identify the next product for replenishment, the system retrieves the root element without scanning the complete inventory.

When a product is restocked, it is removed from the heap, and the remaining elements are reorganized using the Heapify operation. This process ensures that the next lowest-stock product automatically becomes the highest priority.

By using a Min-Heap, the system performs inventory prioritization efficiently even when handling a large number of products.

Python Commands Used

Command	Purpose
`pop()`	Removes the root element.
`return`	Returns the minimum stock product.
`len()`	Determines heap size.

4.6 Time Complexity Analysis

The Min-Heap provides efficient performance for inventory management compared to searching or sorting the entire inventory repeatedly.

Operation	**Time Complexity**
Insert Product	**O(log n)**
Remove Minimum	**O(log n)**
View Minimum Product	**O(1)**
Heap Creation	**O(n)**

The ability to retrieve the product with the lowest stock in constant time makes the Min-Heap highly suitable for inventory management systems.

CHAPTER 5

TASK 3: BUSINESS ANALYTICS AND DATA VISUALIZATION

5.1 Introduction

After completing the ETL process and inventory prioritization, the cleaned dataset is analyzed to extract meaningful business insights. Business analytics helps organizations understand sales performance, customer purchasing patterns, regional demand, and suspicious transactions. Instead of manually examining thousands of transaction records, analytical techniques summarize the data into reports and visualizations that support quick and informed decision-making.

In this project, the processed sales data is analyzed to generate product-wise summaries, revenue reports, regional sales distribution, and fraud analysis. The results are then presented through charts and an interactive dashboard, enabling users to monitor business performance efficiently.

5.2 Subtask 1 – Product Sales Analysis

The first stage of business analytics focuses on analyzing the sales performance of individual products. The cleaned dataset is grouped based on product names, and important metrics such as total quantity sold, total revenue generated, and average sales are calculated. This analysis helps identify the most popular products and those with lower sales, enabling businesses to make better inventory and marketing decisions.

The summarized information is stored in a product summary file, which is later displayed on the dashboard for quick reference.

Python Commands Used

Command	Purpose
groupby()	Groups records by product name.
sum()	Calculates total quantity and revenue.
sort_values()	Sorts products based on sales performance.
to_csv()	Exports the product summary.

5.3 Subtask 2 – Revenue Analysis

Revenue analysis is performed to evaluate the overall financial performance of the business. The system calculates the total revenue generated from all sales transactions and identifies products contributing the highest revenue. This information allows businesses to understand which products generate maximum profit and supports better pricing and sales strategies.

The calculated revenue values are also used to create graphical reports for easier interpretation.

Python Commands Used

Command	Purpose
sum()	Calculates total revenue.
mean()	Calculates average sales value.
sort_values()	Arranges products based on revenue.

5.4 Subtask 3 – Regional Sales Analysis

Regional sales analysis helps identify how sales are distributed across different geographical regions. The processed dataset is grouped based on region, and the total revenue generated from each location is calculated. This enables businesses to identify high-performing markets and regions that require additional promotional activities.

To improve data interpretation, the regional sales information is also displayed through an interactive map, providing a visual representation of sales distribution.

Python Commands Used

Command	Purpose
groupby()	Groups sales data by region.
sum()	Calculates total regional sales.
folium.Map()	Creates an interactive sales map.
folium.Marker()	Marks regional sales locations.

5.5 Subtask 4 – Fraud Detection

The final stage of analytics focuses on identifying suspicious transactions. Transactions are evaluated based on predefined business rules such as unusually high discounts or abnormal purchasing patterns. If a transaction satisfies these conditions, it is flagged as potentially fraudulent for further review.

This analysis helps improve transaction monitoring and enables businesses to identify unusual activities before they impact revenue or customer trust.

Python Commands Used

Command	Purpose
apply()	Applies fraud detection logic to each record.
lambda	Creates conditional fraud rules.
value_counts()	Counts fraudulent and normal transactions.

5.6 Subtask 5 – Data Visualization

To make analytical results easier to understand, the processed data is presented through graphical

visualizations. Bar charts are used to compare product sales and revenue, while an interactive regional map displays geographical sales distribution. These visualizations enable users to identify trends, compare business performance, and make informed decisions without examining raw data. The generated charts and maps are integrated into the project dashboard, providing a user-friendly interface for monitoring business performance.

Python Commands Used

Command	**Purpose**
matplotlib.pyplot	Generates graphs and charts.
plt.bar()	Creates bar charts.
plt.title()	Adds chart title.
plt.xlabel()	Labels the x-axis.
plt.ylabel()	Labels the y-axis.
plt.savefig()	Saves charts as image files.

CHAPTER 6

TASK 4: BACKEND API AND INTERACTIVE DASHBOARD

6.1 Introduction

The final stage of the project focuses on presenting the processed data to users through a backend service and an interactive dashboard. While the ETL pipeline prepares the data and the analytics module generates business insights, users require a simple interface to access this information. To achieve this, a FastAPI backend is used to provide REST API endpoints, and an interactive dashboard is developed using HTML, CSS, and JavaScript.

The backend acts as a bridge between the processed dataset and the frontend application by responding to client requests with structured JSON data. The dashboard retrieves this information through API calls and displays it in the form of tables, charts, and maps. This architecture enables users to monitor sales performance, inventory status, regional sales, and fraud detection results in a user-friendly manner.

6.2 Subtask 1 – FastAPI Backend Development

The backend of the project is developed using the **FastAPI** framework. It manages communication between the processed dataset and the dashboard by exposing multiple REST API endpoints. These endpoints provide information such as product summaries, revenue analysis, inventory status, regional sales, and fraud detection results.

FastAPI was selected because it is lightweight, fast, and automatically generates API documentation, making it suitable for modern web applications.

Python Commands Used

Command	Purpose
FastAPI()	Creates the FastAPI application.
@app.get()	Defines GET API endpoints.
return	Sends JSON response to the client.
uvicorn.run()	Starts the backend server.

6.3 Subtask 2 – API Integration

After developing the backend, API endpoints are connected to the frontend dashboard. Whenever a user opens the dashboard, the frontend sends requests to the backend, which retrieves the required data and returns it in JSON format. This allows the dashboard to display updated business information without directly accessing the dataset.

The API-based architecture also makes the system modular, allowing the backend and frontend to

be developed and maintained independently.

Python Commands Used

Command	Purpose
to_json()	Converts processed data into JSON format.
json.load()	Reads JSON data for API responses.
JSONResponse	Returns structured JSON data to the frontend.

6.4 Subtask 3 – Interactive Dashboard

The dashboard provides a graphical interface for viewing business insights generated during the previous tasks. It displays important information such as product summaries, revenue reports, inventory status, fraud detection results, and regional sales distribution.

The dashboard is developed using **HTML**, **CSS**, and **JavaScript**, making it simple, responsive, and easy to use. Users can quickly understand business performance through visual elements instead of reading large tables of data.

Technologies Used

Technology	Purpose
HTML	Creates the webpage structure.
CSS	Designs and styles the dashboard.
JavaScript	Loads and updates data dynamically.
JSON	Transfers data between backend and frontend.

6.5 Subtask 4 – Dashboard Visualization

To improve data interpretation, various visual components are integrated into the dashboard. Sales performance is displayed using bar charts, product summaries are shown in tables, and regional sales are visualized using an interactive map. These visualizations provide a clear understanding of business trends and support faster decision-making.

The dashboard updates automatically whenever new processed data is available through the backend APIs.

Tools Used

Tool	Purpose
Matplotlib	Generates charts and graphs.
Folium	Creates the interactive regional sales map.
HTML Dashboard	Displays business insights.

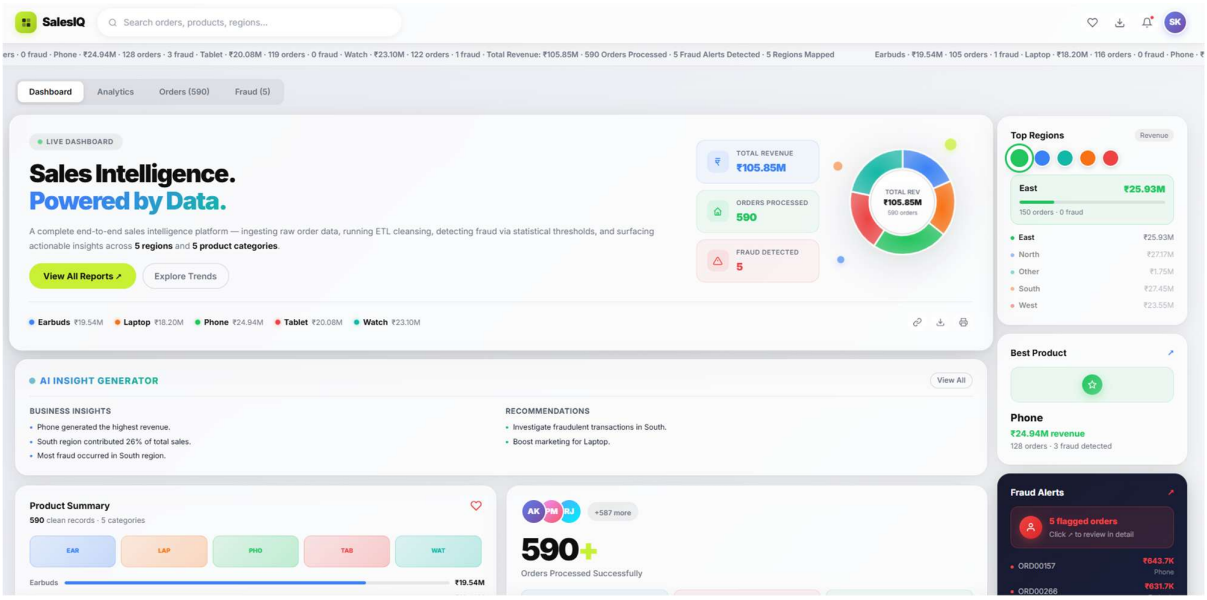
CHAPTER 7

RESULTS

7.1 Introduction

The successful implementation of the proposed system resulted in a complete e-commerce sales intelligence platform capable of processing raw sales data, managing inventory, performing business analytics, and displaying insights through an interactive dashboard. Each module was executed successfully, producing accurate and meaningful outputs that support business decision-making.

The ETL pipeline generated a clean and structured dataset, the Min-Heap algorithm efficiently prioritized inventory, business analytics produced valuable reports, and the FastAPI backend successfully delivered data to the dashboard. The final outputs demonstrate the effectiveness of integrating data engineering, data structures, analytics, and visualization into a single system.



7.2 ETL Pipeline Output

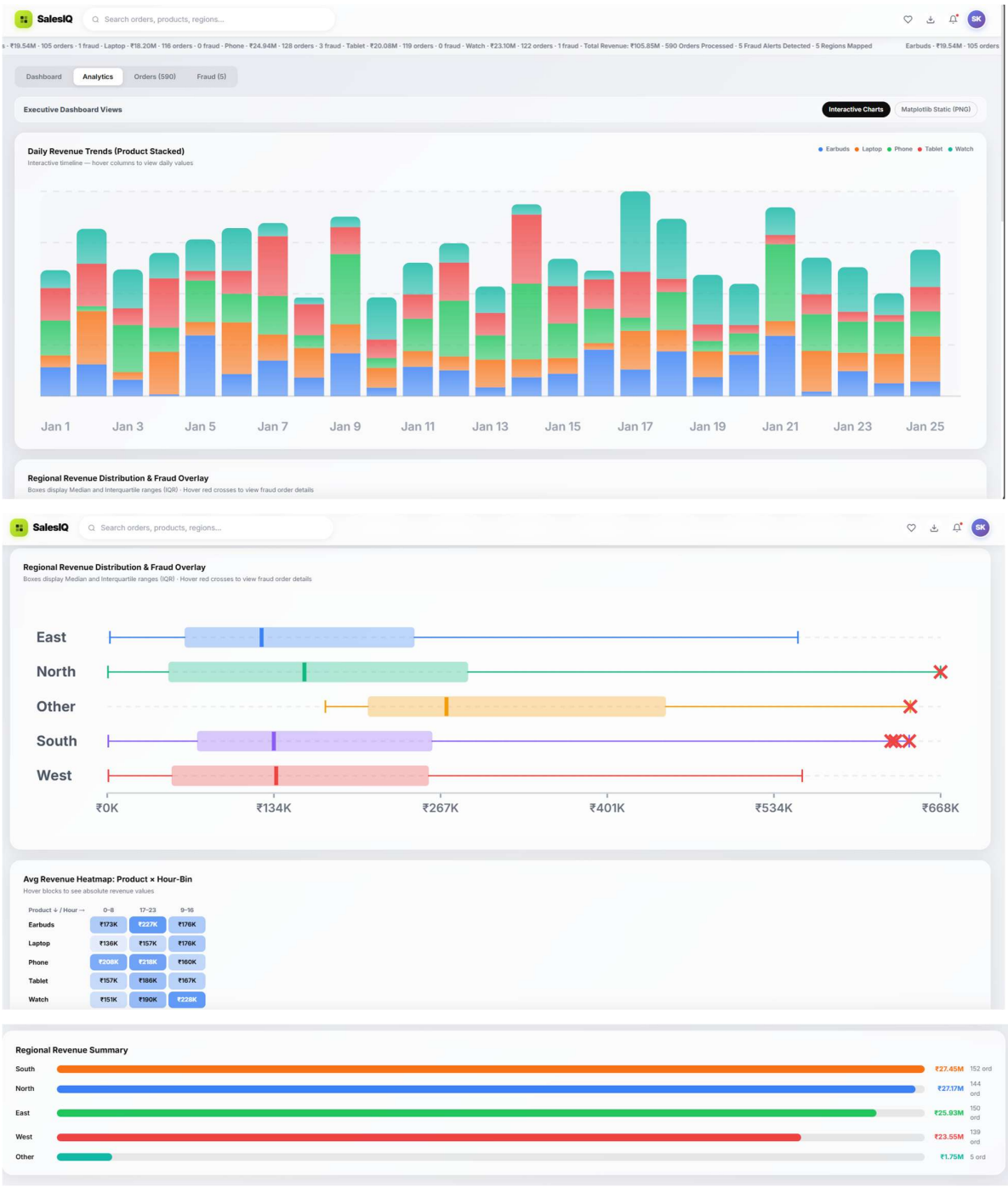
The ETL pipeline successfully transformed the raw dataset into a clean and structured format. Missing values were handled, duplicate records were removed, inconsistent currency formats were standardized, and invalid discount values were corrected. Additional business features such as Total Sales and Net Revenue were also generated during the transformation stage.

The processed dataset was exported in both CSV and JSON formats, making it suitable for further analysis and dashboard integration.

Output Generated:

- Cleaned Sales Dataset
- Product Summary CSV

- Dashboard JSON File



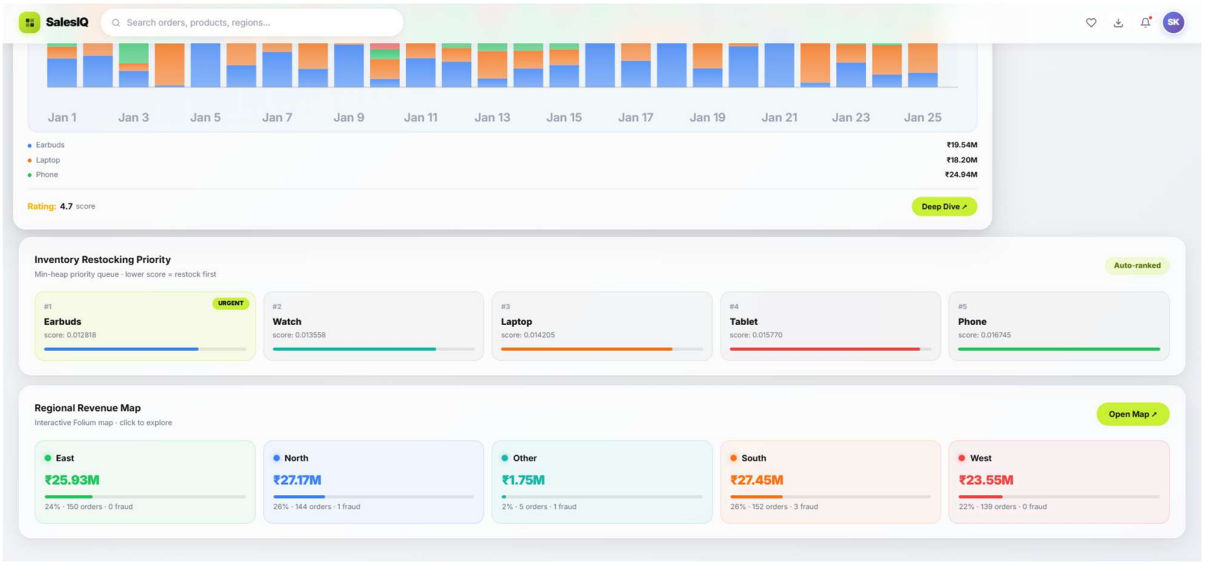
7.3 Inventory Management Output

The Min-Heap algorithm successfully prioritized products based on their available stock quantities. Products with the lowest stock were automatically placed at the highest priority, allowing quick identification of items requiring replenishment.

The implementation reduced the time required to retrieve low-stock products compared to searching the complete inventory manually.

Output Generated:

- Prioritized Inventory List
- Low Stock Product Identification



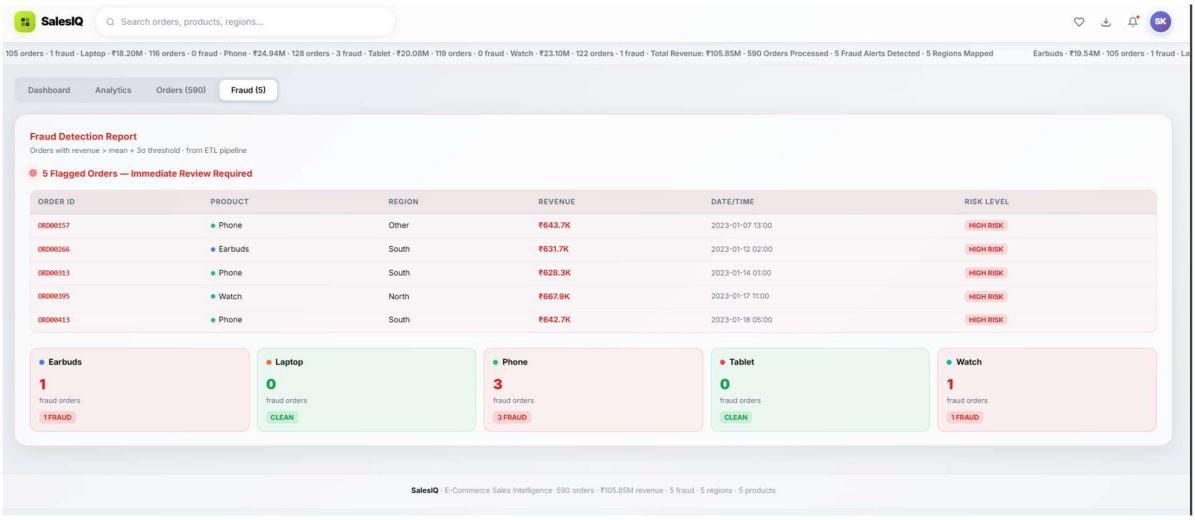
7.4 Business Analytics Output

The analytics module generated meaningful business insights from the processed dataset. Product-wise sales summaries, revenue analysis, regional sales distribution, and fraud detection reports were successfully produced.

These reports enable users to evaluate business performance, identify top-selling products, detect suspicious transactions, and compare sales across different regions.

Output Generated:

- Product Summary Report
- Revenue Analysis
- Regional Sales Report
- Fraud Detection Report



7.5 Dashboard Output

The processed data was successfully integrated with the FastAPI backend and displayed through the interactive dashboard. The dashboard presents business information using tables, charts, and maps, enabling users to monitor sales performance in a simple and visually appealing manner.

Users can easily access product details, inventory status, revenue statistics, fraud reports, and regional sales information from a single interface.

Output Generated:

- Sales Dashboard
- Revenue Charts
- Product Summary Table
- Regional Sales Map